BIG DATA · UDEM

Big Data e Ingeniería de Datos

Clase 3 · Responder bien y guardar para que el total cuadre

Posgrado · UdeM · 9 nov 2026

5 veces

Al unir pedidos con líneas, SUM(O_TOTALPRICE) pasó de 226.8 a 1 134.4 mil millones (unidades monetarias de TPC-H) sin aviso. Rápido e incorrecto.

Clase 2, SF1 · Snowflake, 3 nov. 2026

Otro límite: GROUP BY borra los pedidos a comparar; 1 500 000 quedan en 99 996 filas

```sql
SELECT O_CUSTKEY, COUNT(*) AS PEDIDOS
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
GROUP BY O_CUSTKEY
ORDER BY PEDIDOS DESC, O_CUSTKEY;
```

Una fila por cliente; fecha y monto de cada pedido ya no están. Orden: más pedidos primero y, en empate, menor cliente. Primero: 3451, con 41.

Tres problemas: comparar filas sin perderlas, encadenar cálculos sin errores, guardar para que el total cuadre

1 · Comparar filas sin perderlas

Síntoma: GROUP BY deja 99 996 de 1 500 000 filas

Idea: función de ventana

Slides 6-11 · Lab 3A

→

2 · Encadenar cálculos sin errores

Síntoma: el promedio de promedios se desvía 6.9 %

Idea: QUALIFY, CTE, agregar antes de unir

Slides 18-22 · Lab 3B

→

3 · Guardar para que el total cuadre

Síntoma: una pregunta por región pide 4 uniones

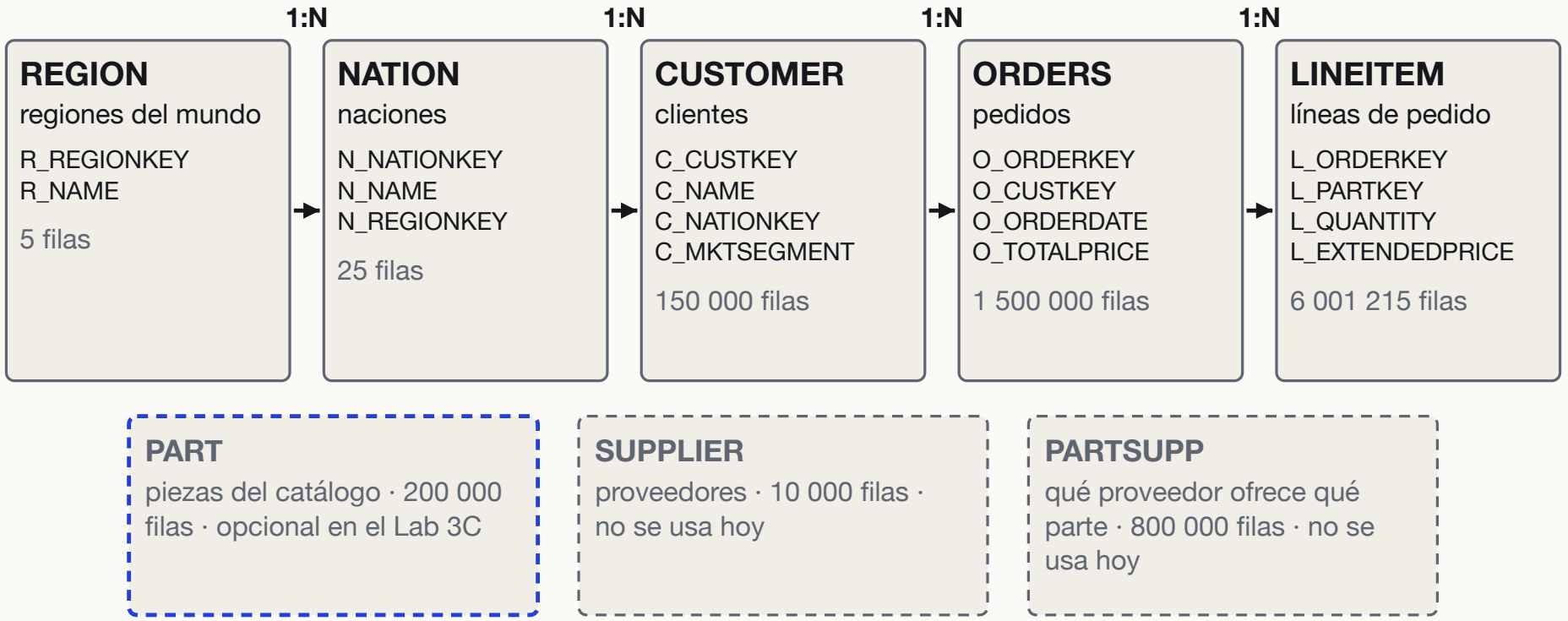
Idea: modelo dimensional con grano

Slides 24-36 · Lab 3C

Caso de cierre: ¿la lluvia cambia los viajes en NYC? (slide 37)

Cada problema trae un síntoma medido y una idea de ingeniería.

Los tres problemas se practican sobre TPC-H: ocho tablas de un mayorista, ya cargadas en tu cuenta



LINEITEM por escala: SF1 6 001 215 · SF10 59 986 052 · SF100 600 037 902

Normalizadas: cada dato vive en una tabla, salvo O_TOTALPRICE (derivado). SF1: factor de escala 1.

Primer problema: una ventana calcula sobre un grupo sin borrar filas; los 41 pedidos siguen ahí

```
SELECT O_ORDERKEY, O_ORDERDATE, O_TOTALPRICE,
       SUM(O_TOTALPRICE) OVER (PARTITION BY O_CUSTKEY) AS TOTAL_CLIENTE
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE O_CUSTKEY = 102022
ORDER BY O_ORDERDATE, O_ORDERKEY;
```

OVER define la ventana: las filas que mira cada fila. PARTITION BY la limita a su cliente. TOTAL_CLIENTE suma O_TOTALPRICE (total del pedido con impuesto): 6 273 788.41.

PARTITION BY agrupa; ORDER BY ordena y convierte el total en acumulado

Monto, cliente 102022	Sin ORDER BY	Con ORDER BY
144 930.46	6 273 788.41	144 930.46
111 485.66	6 273 788.41	256 416.12
63 555.44	6 273 788.41	319 971.56

Ordenados por pedidos y cliente, tres con 41 pedidos empatan: RANK salta, DENSE_RANK no, ROW_NUMBER nunca repite

Cliente	Pedidos	ROW_NUMBER	RANK	DENSE_RANK
3451	41	1	1	1
102004	41	2	1	1
102022	41	3	1	1
79300	40	4	4	2

Una ventana también mira atrás: LAG trae la fila anterior y los días entre pedidos son una resta

```
SELECT O_ORDERKEY, O_ORDERDATE,
       DATEDIFF('day',
                LAG(O_ORDERDATE) OVER (ORDER BY O_ORDERDATE, O_ORDERKEY),
                O_ORDERDATE) AS DIAS_DESDE_ANTERIOR
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE O_CUSTKEY = 102022
ORDER BY O_ORDERDATE, O_ORDERKEY;
```

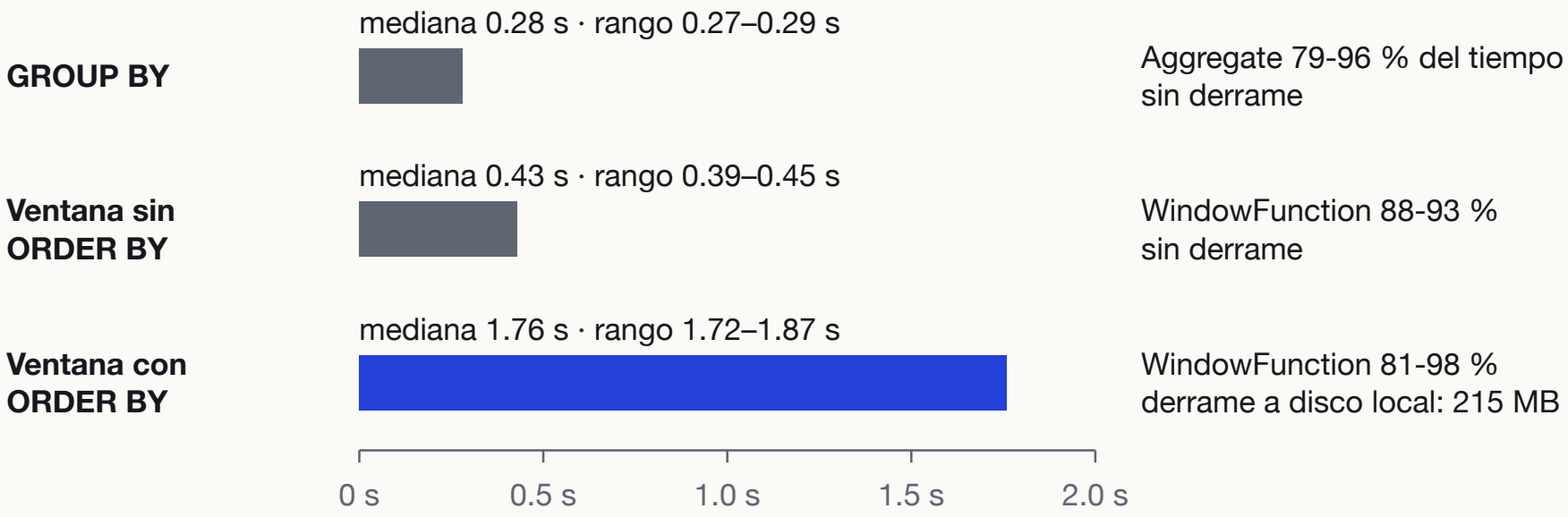
Cliente 102022: el primer pedido da NULL, no tiene anterior. Los siguientes: 49, 44 y 32 días. La mayor pausa es de 227. LEAD mira hacia adelante.

El marco fija cuántas filas mira cada cálculo: una media móvil de tres pedidos con ROWS

```
SELECT O_ORDERKEY, O_ORDERDATE, O_TOTALPRICE,
       AVG(O_TOTALPRICE) OVER (
         ORDER BY O_ORDERDATE, O_ORDERKEY
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS MEDIA_MOVIL_3
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE O_CUSTKEY = 102022
ORDER BY O_ORDERDATE, O_ORDERKEY;
```

Cada pedido promedia sus 2 anteriores y él mismo: 144 930.46, 128 208.06, 106 657.19. Sin marco explícito rige RANGE: dos pedidos del mismo día reciben el mismo valor.

Una ventana con ORDER BY tardó 4 veces más que sin él y derramó 215 MB a disco



Cambio de escala: SF10 (15 M de filas en ORDERS), no SF1 como en las slides 3 a 10.

X-Small, caché de resultados desactivada, 3 corridas por consulta. Señal, no medición fina.

4 veces = 1.76 s con ORDER BY frente a 0.43 s sin él.

Derrame: lo que no cabe en memoria se escribe a disco local.

Lab 3A: con cinco ventanas comparas pedidos sin perder ninguno

Snowsight

1. Si hiciste el bootstrap: Studio, Code Editor, Run space.
2. Pega lab3a.sql (Copy raw file); corre el Paso 0.
3. Resuelve EJ1 a EJ5: top-3, último, caídas, acumulado, participación.
4. Verifica: lab3a_verificacion.sql, Run All.

Las cinco consultas del Lab 3A viven en hojas; un pipeline las necesita en archivos de tu fork

	Snowsight	Code Editor
Dónde vive	En Snowflake	En tu fork
Quién ejecuta	Tú, a mano	Un pipeline: pasos en cadena
Sirve para	Explorar	Repetir

Entras a Snowflake con una llave y a GitHub con tu cuenta; make doctor comprueba Snowflake y AWS



Secrets: valores cifrados de tu fork que Actions lee. Llave: par pública y privada.

Pasos 5 y 6: abres el Code Editor y clonas tu fork

Consola AWS

1. Run space, Open, New Terminal: terminal abierta
2. `git clone`: el prompt es `~/bigdata$`
3. Guía en otra pestaña: `docs/guia.md` de tu fork, pasos 5 a 9: salidas y errores

Pasos 7 a 9: make doctor sin fallos cierra la parada; sigue el descanso

Code Editor

1. Snowsight y terminal: `make snowflake-conexion`, «Conexión configurada»
2. `make setup`: «Setup completado»
3. Si hay AVISO: `gh auth login`, `make secrets`
4. `make doctor`: última línea «Diagnóstico sin fallos»

02

DESCANSO · ABRE EL PROBLEMA 2

El top-3 del Lab 3A salió anidado: una consulta dentro de otra. ¿Cómo se filtra una ventana sin anidar?

Segundo problema, encadenar cálculos: filtrar una ventana obliga a anidar, porque WHERE va antes

```
SELECT O_CUSTKEY, O_ORDERKEY, O_TOTALPRICE,
       ROW_NUMBER() OVER (PARTITION BY O_CUSTKEY
                          ORDER BY O_TOTALPRICE DESC) AS RN
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE RN <= 3;
```

Da error. Orden de evaluación: FROM, WHERE, GROUP BY, HAVING, ventanas, QUALIFY, DISTINCT, ORDER BY, LIMIT. En WHERE la ventana todavía no existe.

QUALIFY filtra después de calcular la ventana: el top-3 por cliente deja de necesitar una subconsulta

```sql
SELECT O_CUSTKEY, O_ORDERKEY, O_TOTALPRICE
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
QUALIFY ROW_NUMBER() OVER (
          PARTITION BY O_CUSTKEY
          ORDER BY O_TOTALPRICE DESC, O_ORDERKEY) <= 3
ORDER BY O_CUSTKEY, O_TOTALPRICE DESC;
```

Devuelve 299 831 filas: hasta 3 pedidos por cliente, porque algunos clientes tienen menos. O_ORDERKEY desempata, porque ROW_NUMBER no fija el orden entre empates.

Una CTE nombra cada paso de la cadena y se lee de arriba abajo

```
WITH por_cliente AS (
  SELECT O_CUSTKEY, COUNT(*) AS PEDIDOS
  FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
  GROUP BY O_CUSTKEY)
SELECT C.C_MKTSEGMENT, SUM(P.PEDIDOS) AS PEDIDOS
FROM por_cliente P JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER C
  ON C.C_CUSTKEY = P.O_CUSTKEY
GROUP BY C.C_MKTSEGMENT;
```

CTE (expresión de tabla común): consulta con nombre, no guarda datos. por_cliente deja 99 996 filas; la unión con CUSTOMER suma PEDIDOS por segmento: 297 453 en AUTOMOBILE.

Agregar las líneas antes de unir no repite O_TOTALPRICE; la ejecución no cambió

Medida (SF10, X-Small)	Unir y agregar	Agregar y unir
Filas del Join LINEITEM-ORDERS	60 M	15 M
Ejecución	0.9-1.3 s	1.1-1.2 s
Total O_TOTALPRICE	5.0 veces el real	exacto

Otra trampa: con medias desiguales, el promedio de promedios se desvía 6.9 %

Estado	Pedidos	Promedio
F	729 413	150 398.22
O	732 044	150 288.47
P	38 543	184 446.39
Promedio de promedios		161 711.03
Directo = ponderado (SUM/SUM)		151 219.54

Lab 3B: QUALIFY evita anidar tres consultas y el promedio ponderado corrige un error

Snowsight

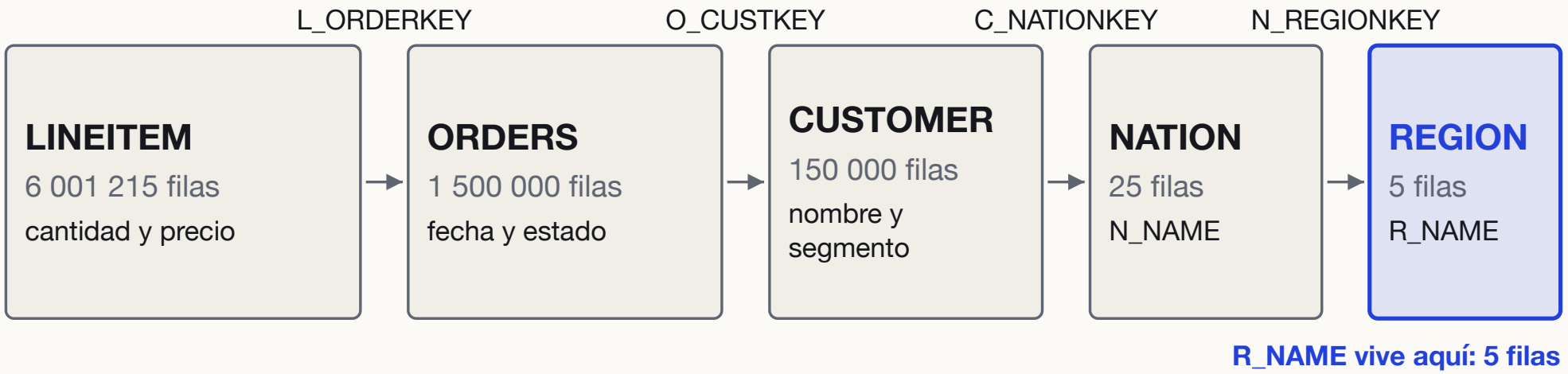
1. Pega lab3b.sql (Copy raw file); corre el Paso 0.
2. B1 a B3: EJ1 a EJ3 con QUALIFY.
3. B4: usa el promedio ponderado.
4. Corre lab3b_verificacion.sql con Run All.

Tercer problema, guardar para que el total cuadre: preguntar por región pide cuatro uniones cada vez

```
SELECT R.R_NAME, SUM(L.L_EXTENDEDPRICE * (1 - L.L_DISCOUNT)) AS INGRESO
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM L
JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS O ON L.L_ORDERKEY = O.O_ORDERKEY
JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER C ON O.O_CUSTKEY = C.C_CUSTKEY
JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.NATION N ON C.C_NATIONKEY = N.N_NATIONKEY
JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.REGION R ON N.N_REGIONKEY = R.R_REGIONKEY
GROUP BY R.R_NAME ORDER BY 1;
```

Cuatro uniones para traer un nombre. Si la dimensión (lo que describe) trajera cliente, nación y región, bastaría una unión con el hecho (lo que se mide).

Normalizar guarda cada dato una vez: R_NAME vive en 5 filas, no en 6 M



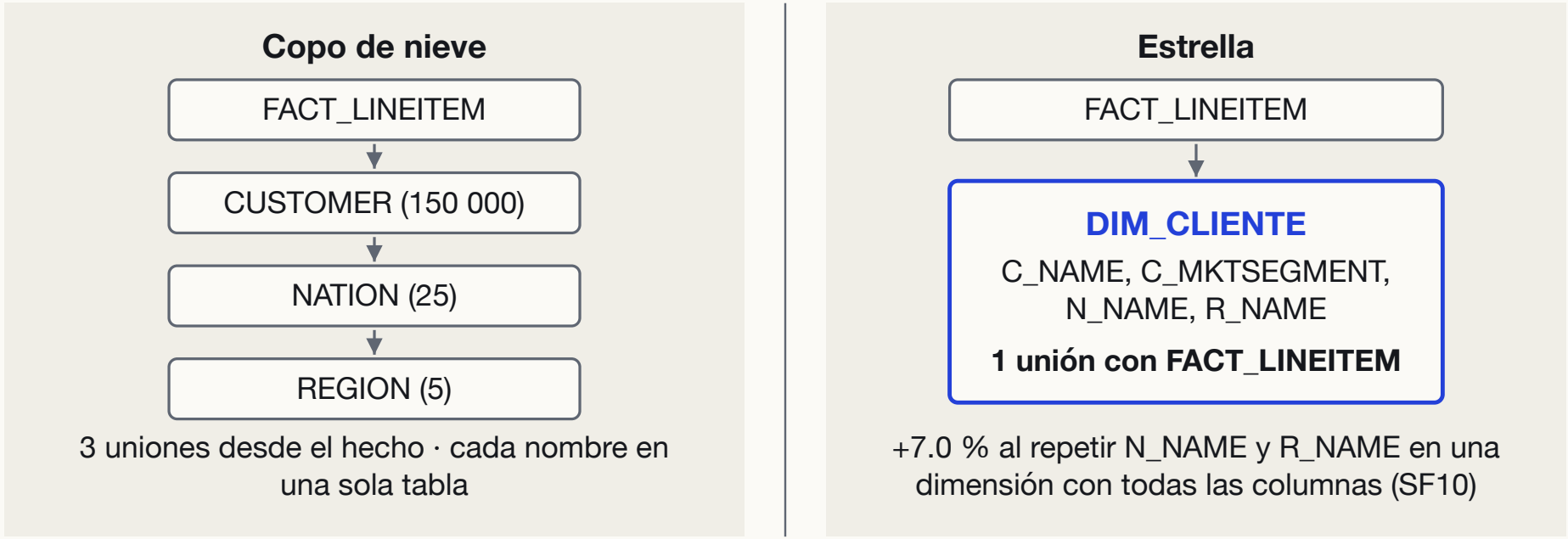
Si R_NAME estuviera en cada línea: 6 001 215 copias

Forma normal 1: un valor por celda. 2 y 3, simplificado: cada columna depende de su llave.

Desnormalizar (repetir datos) redujo la ejecución casi a la mitad; hecho y dimensión ocuparon 6-7 % más

SF10, 3 corridas	Uniones	Ejecución (s)	Dimensión (MB)
Normalizado	4	1.2-1.3	108.3
Hecho + 3 tablas	3	0.65-0.69	108.3
Hecho + dimensión	1	0.60-0.65	115.9

Dos diseños: copo de nieve, lo medido más 3 tablas; estrella, lo medido más una dimensión



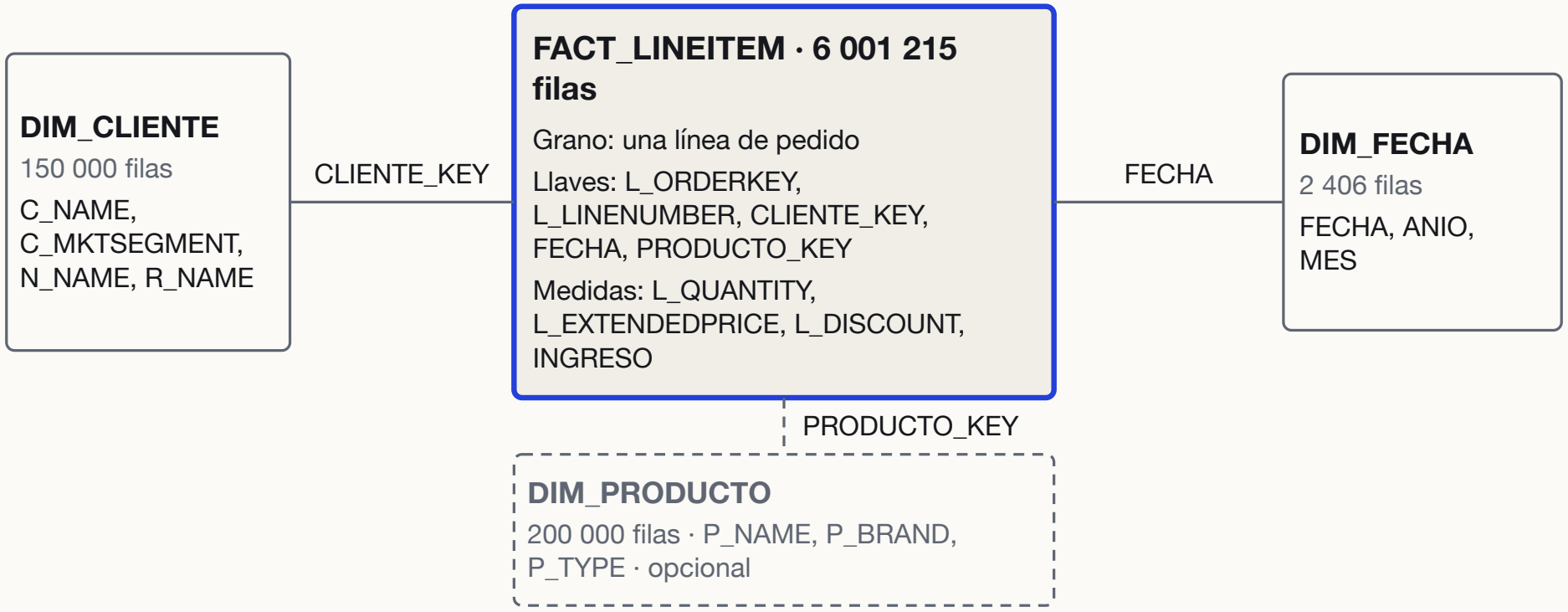
Ambos usan FACT_LINEITEM con O_CUSTKEY copiado: +6.2 % de espacio (88 MB) en el hecho.

Estrella: dimensión en una tabla. Copo: dimensión dividida. CUSTOMER, NATION y REGION son un copo.

El grano declara qué es una fila: en FACT_LINEITEM (hechos), una línea de pedido

Grano	Una fila es	Filas SF1
Línea de pedido	línea de LINEITEM	6 001 215
Pedido	fila de ORDERS	1 500 000
Día	día del rango	2 406

FACT_LINEITEM guarda lo que se mide y las dimensiones describen cliente, fecha y producto



Hecho transaccional: una fila por evento. SF1: hechos 6 001 215; clientes 150 000; fechas 2 406.

Las aditivas se suman por cualquier dimensión; promedios y lluvia diaria, no

Medida	Tipo	Cómo se agrega
INGRESO	Aditiva	SUM por cualquier dimensión
Precio promedio	No aditiva	Guardar SUM y COUNT
LLUVIA_MM del gold (tabla final)	Semiaditiva	No sumar por zona

Grano mezclado: la medida del pedido, repetida en cada línea, infla la suma 5 veces

Un pedido (uno) tiene varias líneas (muchos): al unir, el pedido se repite por cada línea.

ORDERS
pedido A · O_TOTALPRICE 100

LINEITEM
A-1
A-2
A-3

JOIN →

Resultado del JOIN
A-1 · **100**
A-2 · **100**
A-3 · **100**

SUM(O_TOTALPRICE) = 300; el pedido vale 100

1 500 000 → 6 001 215

filas: pedidos contra filas tras el JOIN (× 4.0)

226.8 → 1 134.4 mil millones

SUM(O_TOTALPRICE) sin JOIN contra con JOIN (× 5.0)

El monto crece más que las filas porque los pedidos grandes tienen más líneas y se repiten más.

SUM(O_TOTALPRICE) pasa de 226.8 a 1 134.4 mil millones: 1.5 M pedidos son 6.0 M filas.

La fecha también es dimensión: el cliente 102022 compró en 39 de 2 406 días

```sql
WITH r AS (SELECT MIN(O_ORDERDATE) AS MINF, MAX(O_ORDERDATE) AS MAXF
           FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS),
d AS (SELECT DATEADD('day', ROW_NUMBER() OVER (ORDER BY SEQ4()) - 1, MINF)
             AS FECHA, MAXF
      FROM r, TABLE(GENERATOR(ROWCOUNT => 3000))
      QUALIFY FECHA <= MAXF),
p AS (SELECT DISTINCT O_ORDERDATE AS FECHA
      FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS WHERE O_CUSTKEY = 102022)
SELECT COUNT(*) AS DIAS, COUNT(p.FECHA) AS CON_PEDIDOS,
       COUNT(*) - COUNT(p.FECHA) AS SIN_PEDIDOS
FROM d LEFT JOIN p ON p.FECHA = d.FECHA;
```

GENERATOR crea 3 000 filas; QUALIFY deja 2 406 días. LEFT JOIN conserva 2 367 sin pedidos. Los 41 pedidos caen en 39 días: dos días tienen dos.

SCD tipo 1 sobrescribe: si el cliente 102022 se mudara, 41 pedidos pasarían de ASIA a EUROPE

Antes

C_CUSTKEY	N_NAME	R_NAME
102022	INDONESIA	ASIA

UPDATE →

Después

C_CUSTKEY	N_NAME	R_NAME
102022	**FRANCE**	**EUROPE**

Ingreso del cliente 102022 por región (41 pedidos, misma escala)



ASIA pierde 6 039 653.63 y EUROPE lo gana; ningún pedido cambió

Ingreso = suma de L_EXTENDEDPRICE × (1 − L_DISCOUNT), sin impuesto; O_TOTALPRICE (6 273 788.41) sí lo incluye.

Mudanza supuesta; pedidos reales de TPC-H SF1. Valores en unidades monetarias de TPC-H.

SCD: dimensión de cambio lento; el tipo decide qué pasa con la historia.

Mudanza supuesta, SCD tipo 2: por fecha del pedido, 19 van a ASIA y 22 a EUROPE

Versión (clave sustituta)	Nación (cliente 102022)	Región	Vigente desde	Hasta	Pedidos
1	INDONESIA	ASIA	1992-01-01	1995-05-31	19
2	FRANCE	EUROPE	1995-06-01	9999-12-31 (sin fin)	22

Lab 3C: construyes la estrella y compruebas que cuadra

Snowsight

1. Pega lab3c.sql; corre Pasos 0-1.
2. Crea DIM_CLIENTE; corre DIM_FECHA.
3. Carga FACT_LINEITEM.
4. Mide cuál paso de tu carga costó más y abre su Query Profile.
5. Ingreso por región y año.
6. Corre lab3c_verificacion.sql.

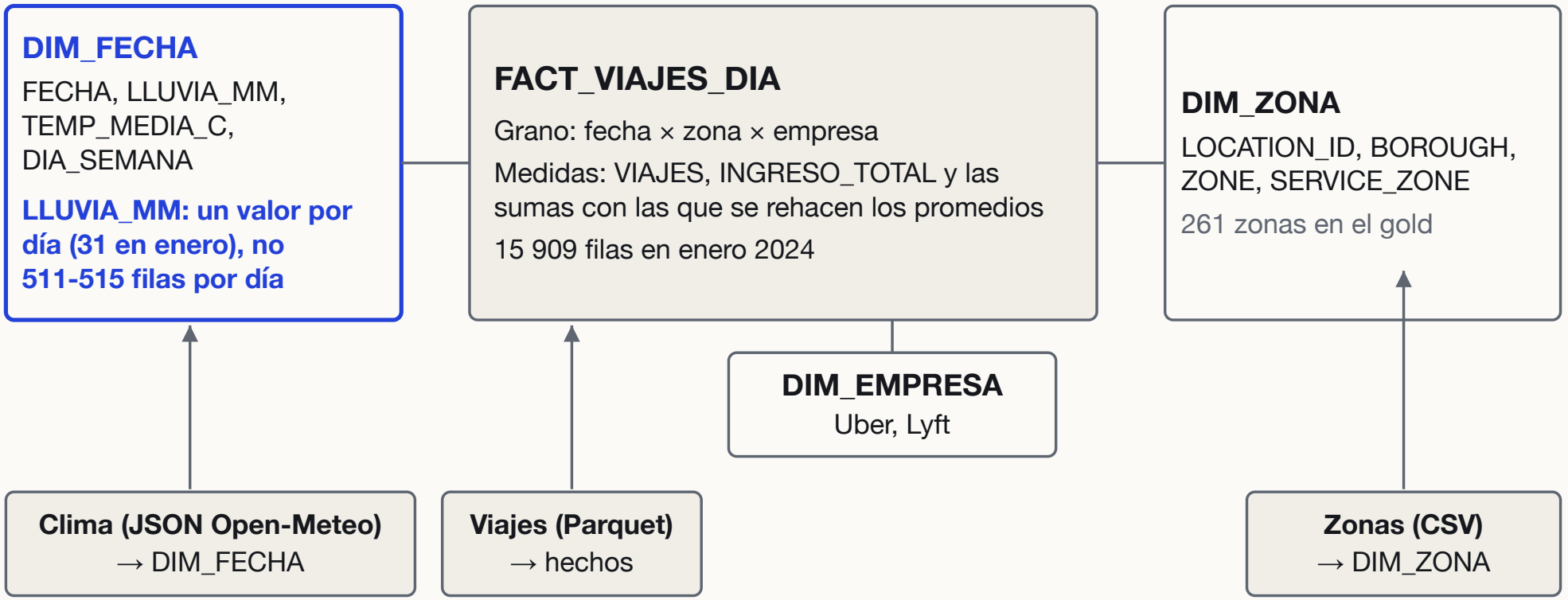
Una estrella cuadra cuando pasan cuatro controles: filas, grano, claves sin pareja e ingreso

```sql
SELECT 'filas' AS CONTROL, COUNT(*) AS FACT, 6001215 AS ESPERADO
FROM FACT_LINEITEM
UNION ALL
SELECT 'grano', COUNT(DISTINCT L_ORDERKEY * 10 + L_LINENUMBER), 6001215
FROM FACT_LINEITEM
UNION ALL
SELECT 'sin cliente', COUNT(*), 0
FROM FACT_LINEITEM F LEFT JOIN DIM_CLIENTE D USING (CLIENTE_KEY)
WHERE D.C_NAME IS NULL
UNION ALL
SELECT 'ingreso', ROUND(SUM(INGRESO)), 218102223885 FROM FACT_LINEITEM;
```

Un control dice si el dato es correcto; el tiempo por paso dice cuánto costó obtenerlo. Origen: 6 001 215 filas y claves; 0 sin cliente; ingreso 218 102 223 885.

Caso NYC, lluvia y viajes: la estrella propuesta guardaría LLUVIA_MM en 31 filas, no en 15 909

Propuesta de diseño; el gold actual (tabla final del pipeline) repite LLUVIA_MM en cada fila



LLUVIA_MM: lluvia del día en mm. Hoy se repite en cada fila del gold.

Tres problemas resueltos; la clase 4 lleva 239.5 M de viajes de NYC a un lake

1 · Comparar filas sin perderlas

Quedó:
ventana: el detalle y el total en la misma fila

Slides 6-11 · Lab 3A

→

2 · Encadenar cálculos sin errores

Quedó:
filtrar sin anidar; pasos con nombre; promedio ponderado (SUM / SUM) 151 219.54, igual al directo

Slides 18-22 · Lab 3B

→

3 · Guardar para que el total cuadre

Quedó:
de 4 uniones a 1; ejecución casi a la mitad en esta medición; 4 controles

Slides 24-36 · Lab 3C

Clase 4: lake en S3 y carga a Snowflake de los 239.5 M de viajes de NYC (2024)

¿La lluvia cambia los viajes? La responde el Taller 1, entrega 15 de noviembre.

Lake: almacén de archivos. La pregunta de la lluvia la responde el Taller 1, 15 nov.